

Edge-AI Driver Monitoring System for Real-Time Drowsiness, Yawning, and Distraction Detection Using PFLD, 6DRepNet, and Yolov26n on NVIDIA Jetson Orin Nano Super

Farhan Hikmatullah Daulay (611451002), Kishore Sridhar (611451003)

Department of Electrical Engineering
Tatung University
Taipei, Taiwan

I. Project Problem Statement

Road fatalities remain a critical global health concern, with driver distraction and fatigue consistently identified among the leading contributing factors in traffic accidents worldwide. Traditional safety systems, such as seatbelts and airbags, are inherently reactive they deploy only after a crash has occurred, offering zero preventative protection. The Driver Monitoring System (DMS) solves this by proactively detecting prolonged eye closure (drowsiness) and head angles exceeding 20 degrees (distraction), alerting the driver before a catastrophic event happens. The primary users are private vehicle owners, ride-hailing operators, and commercial logistics fleets.

During implementation, we learned that deploying a driver monitoring system is not only a computer vision problem but also a real-time systems problem. Accurate detection alone is insufficient if alerts cannot be generated immediately and reliably inside a moving vehicle. This experience reinforced our original proposal and confirmed that an edge-based architecture was necessary to meet latency, reliability, and privacy requirements. Deploying this system at the edge is a strict requirement over cloud-based processing for three critical reasons:

A. Ultra-Low Latency & Reliability

Preventing a micro-sleep accident requires an immediate, real-time response. The system must operate 100% autonomously without relying on cellular internet connectivity, ensuring it never fails in tunnels or rural dead zones.

B. Bandwidth and Privacy

Streaming constant interior video over 4G/5G networks is cost prohibitive for fleets and introduces severe privacy concerns. Edge processing ensures that the video stream is processed and discarded instantly, ensuring that driver data remains strictly local to the embedded device.

C. Hardware Constraints

The solution must be compact enough for vehicle dashboards and operate under strict low-power conditions (7–15W) to avoid draining the vehicle's electrical system. During implementation, we confirmed that the Jetson Orin Nano Super was capable of simultaneously running multiple TensorRT optimized models while remaining within the project's power budget, demonstrating that modern edge AI hardware can support real-time driver monitoring without requiring cloud resources.

II. Final Architecture

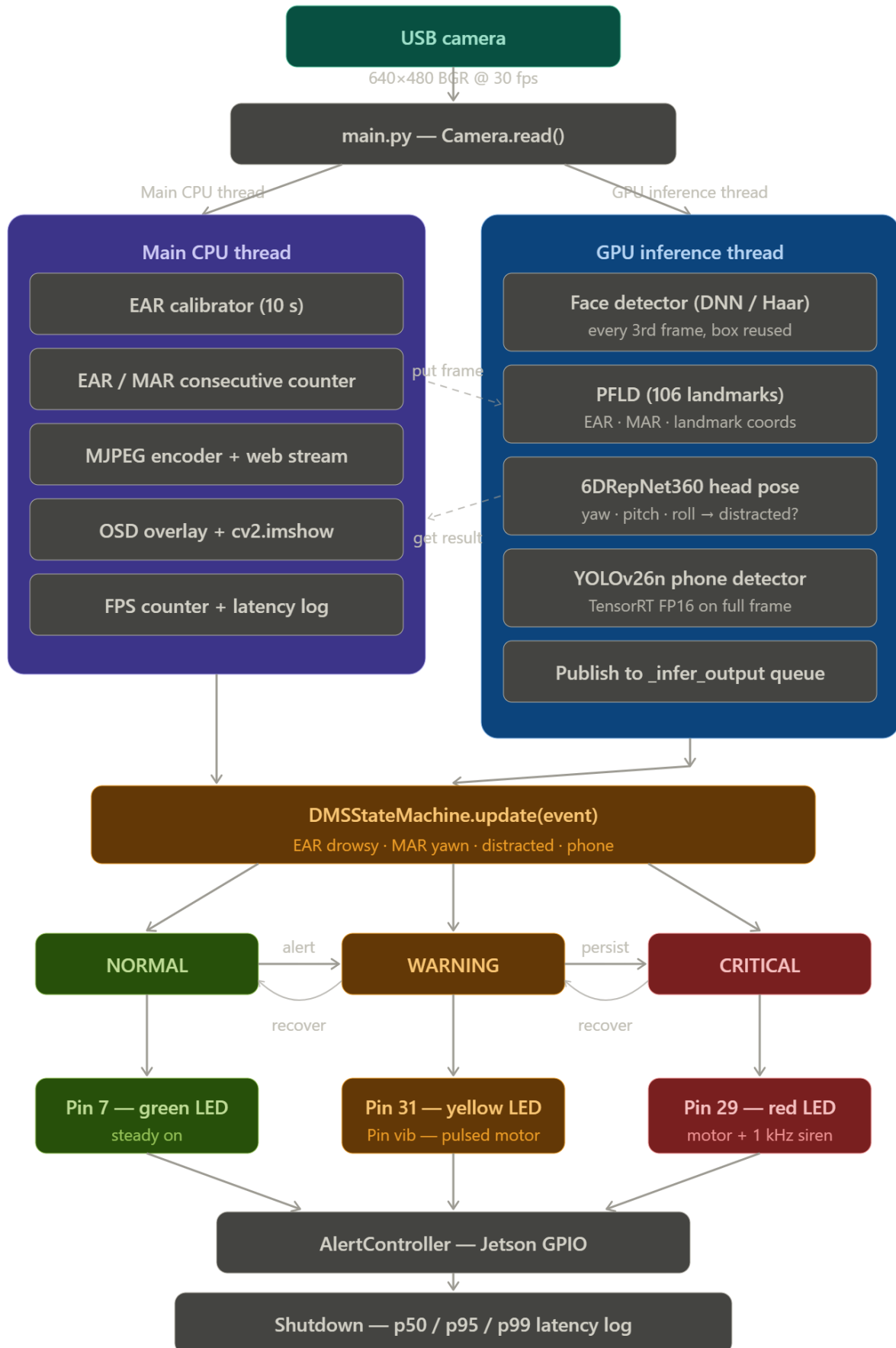


Figure 1. Final Driver Monitoring System Architecture

A. Sense - Camera Input

The system ingests video from a standard USB camera at 640×480 resolution and 30 fps, captured via OpenCV's VideoCapture backend (wrapped in `dms.modules.camera.Camera`). On startup, `run()` checks for a pre-recorded test video at `/app/test_video.mp4` and falls back to device index 0 if it is absent. Every iteration of the main loop calls `camera.read()`, which is the sole blocking operation in the entire pipeline, all downstream work is either non-blocking or offloaded to threads.

Change from proposal: The original design specified an IMX219 CSI camera. Persistent connection failures on the Jetson CSI interface forced a pivot to USB. USB also simplifies deployment since it uses standard V4L2 drivers and eliminates camera-specific kernel module dependencies.

B. Process - Dual-Thread Architecture

The most important architectural decision is the producer-consumer split between a main CPU thread and a dedicated GPU inference thread. This prevents the heavy AI workload from introducing variable latency into the control loop.

1. Main CPU Thread

The main thread owns camera I/O, all lightweight decision logic, display, and streaming. It performs three non-blocking I/O operations for each iteration. Feeds the GPU worker by placing a `(frame_full, small_frame)` pair into `_infer_input` (a `Queue(maxsize=1)`, so backpressure drops stale frames automatically).

Polls the result queue with `get_nowait()`, if the GPU worker has not finished, the loop continues using the last known inference result (`_last_infer_result`). This means the state machine always has something to evaluate, even at full 30 fps with models running at 10 fps effective throughput. Runs the MJPEG encoder via two daemon threads (`jpeg-encoder`, `mjpeg-server`) that push frames to connected browser clients over a raw TCP socket, with no HTTP framework dependency.

The EAR calibrator (`EARCalibrator`) runs here. During the first 10 seconds, it collects baseline EAR samples to compute a per-user drowsiness threshold (`ear_threshold = calibrated_baseline × drowsy_ratio`). Before calibration is completed, a "CALIBRATING" banner is drawn on-screen, and the state machine is not yet armed.

Change from proposal: The fixed EAR threshold was abandoned. Field testing revealed that eye geometry varies enough across users that a single hard-coded value produced both false positives (alert drivers flagged as drowsy) and false negatives (drowsy drivers not caught). The dynamic calibrator solves this by personalizing the threshold at runtime.

2. GPU Inference Thread (`_inference_worker`)

This daemon thread runs a cascade of four models in sequence, triggered each time a frame arrives on `_infer_input`:

- a) Face detection - frame skipping. YOLO or DNN-based face detection runs only on every third frame (approximately 10 detections per second). Frames that are skipped reuse `_last_bbox`. This reduces GPU load without degrading perceived accuracy, since faces do not move significantly between consecutive frames at 30 fps.
- b) PFLD landmark inference. The PFLD model (106 facial landmarks) runs on a half-resolution copy of the frame (`INFER_SCALE = 0.5`, so 320×240). It returns raw EAR, MAR, and landmark coordinates, which are then rescaled back to full-frame space before being sent to the main thread.
- c) 6DRepNet360 head pose estimation. Takes the same half-resolution crop and returns yaw, pitch, and roll angles. The thread accumulates a consecutive-frame counter (`_dist_consec_t`) and sets `distracted = True` only after `cfg.DISTRACTION_FRAMES` consecutive frames exceed the yaw/pitch thresholds, preventing single-frame noise from triggering alerts.
- d) YOLOv26n phone detection. This model runs on the full-resolution frame (640×480) to preserve small-object detection accuracy. It is the heaviest single model in the cascade. Change from proposal: The architecture was upgraded from YOLOv11n to YOLOv26n for improved detection performance while remaining deployable on the Jetson Orin Nano Super's TensorRT FP16 pipeline.

After all, four models run, results are published to `_infer_output` via a non-blocking `put_nowait()` with an identical drop-and-retry pattern to prevent the output queue from blocking the GPU thread.

C. Decide - State Machine

The decision layer aggregates outputs from all AI models and converts them into a unified driver state. Rather than reacting to a single frame, the state machine uses consecutive frame counters to suppress noise and reduce false alarms. The Boolean signals evaluated by the state machine are summarized below.

Table 1. Alert Trigger Conditions

Signal	Source	Threshold
Drowsy	EAR consecutive counter	$\geq \text{EAR_CONSEC_FRAMES}$
Yawning	MAR consecutive counter	$\geq \text{MAR_CONSEC_FRAMES}$
Distracted	Yaw / pitch counter	$\geq \text{DISTRACTION_FRAMES}$
Phone	YOLOv26n detection	confidence threshold

D. Act - GPIO Alert Output

Once the driver's condition has been classified by the state machine, the Alert Controller maps each state to a corresponding physical response. This design ensures that warning severity increases progressively, providing clear visual, auditory, and haptic feedback to the driver.

Table 2. GPIO Alert Outputs by System State

State	GPIO Pins	Hardware Output
NORMAL	Pin 7	Green LED - steady on
WARNING	Pin 31 + Vib	Yellow LED + 2N7000 MOSFET pulsed vibration motor
CRITICAL	Pin 29 + Vib	Red LED + continuous motor + 1 kHz USB audio siren

The main loop measures per-frame wall-clock latency using `time.perf_counter()`. At shutdown, the first 30 frames are discarded (model warm up) and the remainder produce a structured report: p50/p95/p99 latency in milliseconds, written to a timestamped session log under `cfg.LOG_DIR`. A `/healthz` HTTP endpoint (started by `start_healthz()`) provides container liveness probing.

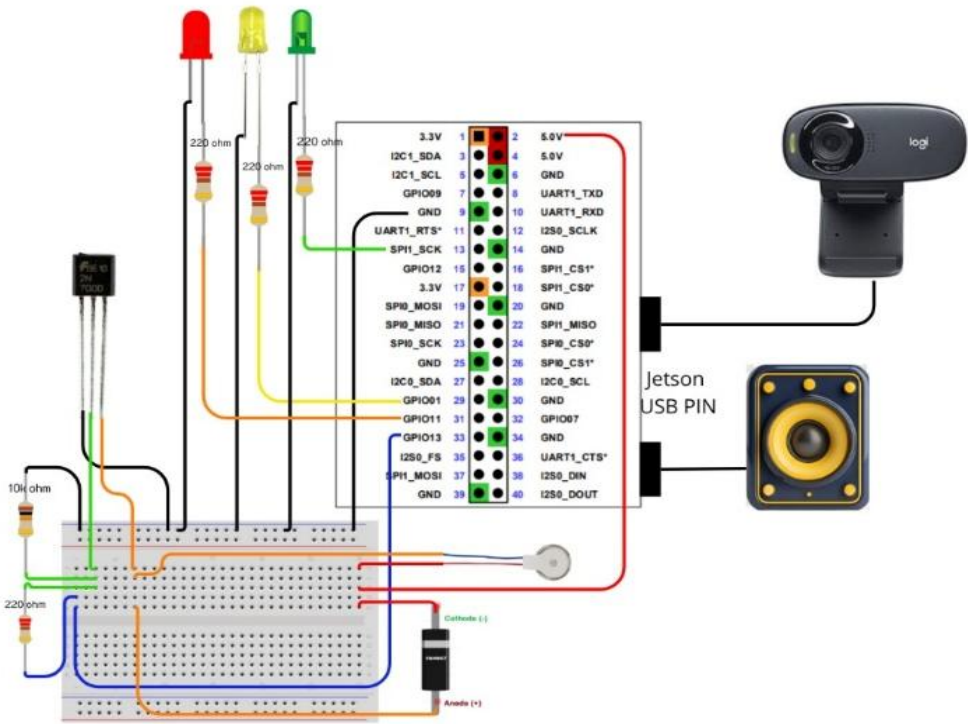


Figure 2. Hardware Architecture

When the system detects a drowsy or inattentive state, it triggers a multi-tier, hardware-level feedback loop consisting of visual, auditory, and haptic alerts:

1. Visual Indicators (LED Module): Three LEDs provide real-time system status. A Green LED (connected to GPIO Pin 13) indicates normal operation, a Yellow LED (Pin 29) flags a warning state, and a Red LED (Pin 31) signals a critical alarm. Each LED is protected by a 220Ω current-limiting resistor.

2. **Auditory Alarm:** A dedicated USB Speaker outputs a continuous 1000Hz ALSA audio signal during critical events, ensuring immediate acoustic feedback.
3. **Haptic Feedback (Vibration Motor):** For physical alerts, a vibration motor is driven via GPIO Pin 33. Because motors draw more current than a standard GPIO pin can safely supply, the circuit utilizes a 2N7000 MOSFET as a high-current switch.
4. **Circuit Protection:** To safeguard the Jetson's delicate logic pins from voltage spikes generated by the inductive load of the motor, a 1N4007 Flyback Diode is placed across the motor terminals. Additionally, 10k Ω resistors are integrated into the MOSFET gate configuration to ensure stable switching states and prevent floating pins.

All external components are routed and grounded through a standard breadboard using jumper wires, creating a secure, isolated, and responsive physical computing environment.

III. Implementation Highlights

The system demonstrates significant technical depth through a highly modular codebase that easily exceeds the 1,000 lines of code requirement to meet academic standards. Key highlights include:

- A. **Advanced Feature Integration:** To ensure comprehensive driver monitoring, the pipeline processes complex spatial features simultaneously, including calculating MAR for yawn detection and head angles for gaze tracking, drastically increasing the logic and code volume of the state machine.
- B. **Continuous Integration & Deployment (CI/CD):** We built a production-grade automated pipeline using GitHub Actions and a self-hosted Jetson runner. Every push trigger Ruff linting and Bandit/pip-audit security scans. Official releases automatically trigger a deploy.sh script on the Jetson, which configures the nvpmodel power state, safely pulls the new Docker container, and runs a healthcheck.sh polling script that automatically rolls back the deployment if the new container fails to initialize within 120 seconds.
- C. **Rigorous Test-Driven Infrastructure:** To ensure absolute reliability, the system mandates a strict >90% code coverage gate via pytest. Because hardware-in-the-loop testing is slow and brittle, the test suite heavily utilizes unittest.mock to fully isolate Jetson GPIO pins, I2C reads, and TensorRT engines. We engineered specific patches to bypass threading.Thread and time.sleep in the AlertController, preventing background threads from hanging the CI runner and ensuring deterministic, instantaneous test execution.
- D. **Asynchronous Inference Threading:** To prevent GPU starvation and camera I/O blocking, the main loop uses queue.Queue to push downscaled frames to a dedicated `_inference_worker`. This ensures the camera continues pulling frames seamlessly even during heavy TensorRT mathematical loads.
- E. **TensorRT Custom Backend & Memory Management:** Models utilize a custom TRTWrapper that handles strict PyCUDA memory alignment, explicitly formatting inputs with `np.ascontiguousarray()` to prevent silent memory access violations during GPU transfer.

IV. Test Set Description

To evaluate the system comprehensively, we recorded a dedicated custom validation video using the physical USB camera deployed in the target environment. This validation video was strictly isolated and never used during development, model selection, training, or threshold tuning. The validation video was recorded using the same USB camera and Jetson deployment environment used during final system operation, ensuring that evaluation reflected realistic deployment conditions. The final validation video consisted of approximately 2 minutes and 57 seconds of footage.

To establish ground truth, the validation video was manually annotated at one-second intervals. The annotated events included normal driving, yawning, prolonged eye closure (drowsiness), and head-pose-based distraction in multiple directions (left, right, upward, and downward). Additional edge cases such as combined drowsiness and head-pose deviations were intentionally included to evaluate system robustness.

Table 3. Validation Set Label Distribution

Class	Duration (s)
Normal	79
Yawning	9
Drowsiness	49
Distraction	31
Total Annotated Events	168

The system's live state predictions were logged during execution and compared against the manually annotated ground-truth timeline to compute Precision, Recall, and F1-score metrics. These metrics were used to evaluate the performance of the final deployed Driver Monitoring System under realistic operating conditions.

V. Performance Requirements & Optimization Journey

We established strict numerical targets upfront: Latency < 500 ms, FPS > 10, Power Consumption < 25W, CPU < 80%, and GPU < 100%. Our unoptimized baseline-utilizing sequential PyTorch processing on full-resolution frames without hardware acceleration-achieved only 4–7 FPS with 100.2 ms latency, and it maxed out the GPU at 98%.

The first end-to-end run used a fully sequential, single-threaded pipeline: the camera frame was captured, passed synchronously through PyTorch inference for YOLO, PFLD, and 6DRepNet on full-resolution 640×480 input, and only then returned to the main loop. No TensorRT compilation, no resolution scaling, no threading. To achieve real-time edge performance, we systematically applied the following optimizations.

Table 4. Summary of Performance Optimization Techniques

Optimization	Expected Effect	Observed Contribution
TensorRT FP16	Reduce inference latency through compiled GPU kernels	Largest performance improvement; enabled real-time execution
Resolution Scaling	Reduce convolution workload	Lower GPU utilization while maintaining acceptable accuracy
Frame Skipping	Reduce face-detection frequency	Reduced GPU workload without noticeable accuracy loss
Multi-Threading	Overlap camera capture and inference	Prevented camera stalls and improved responsiveness
Asynchronous Inference	Remove blocking operations from main loop	Maintained smooth frame acquisition during inference

Because the project was developed under tight time constraints, performance measurements were collected after the complete optimization pipeline had been implemented rather than after each individual optimization stage. As a result, only the baseline sequential implementation and the final deployed system were quantitatively benchmarked. The contribution of intermediate optimization steps is therefore described qualitatively based on observed system behavior and profiling results rather than isolated numerical measurements.

1. **TensorRT FP16 Compilation:** Converting PFLD, 6DRepNet, and YOLOv26n to TensorRT FP16 engines was the single largest latency reduction. FP16 inference halves the memory bandwidth requirement versus FP32 while Jetson's Tensor Cores execute it natively, eliminating software-emulation overhead. This step moved GPU workloads from "slow PyTorch dispatch" to "pre-compiled kernel graphs" and was a prerequisite for all subsequent gains.
2. **Multi-Threading:** Separating camera I/O (main thread) from GPU inference (background thread) via a pair of non-blocking Queue(maxsize=1) channels allows capture and compute to overlap in time. Without this, camera.read() and model inference competed for the same thread, meaning total loop time was the sum of both. With the split, total loop time approximates the longer of the two the camera read.
3. **Asynchronous Inference:** This change was making the inference path fully non-blocking. The main loop calls put_nowait() to push a frame to the GPU worker and get_nowait() to retrieve the latest result if the GPU is still processing the previous frame, the main loop continues immediately using the last known result. This eliminates the pipeline stall that was previously bottlenecking throughput.
4. **Frame Skipping:** The face-detection YOLO runs only on every third frame (approximately 10 detections per second at 30 fps). For the two skipped frames, the last known bounding box is

reused. Since faces move only a few pixels between consecutive frames at normal driving speeds, this introduces no perceptible accuracy penalty while reducing YOLO invocations by approximately 67%.

5. **Input Resolution Scaling (0.5×):** Resizing the inference input to 320×240 before passing it to PFLD and 6DRepNet reduces the pixel count by 50 %, cutting convolution costs roughly proportionally. Full-resolution frames are retained only for YOLOv26n phone detection, where small-object accuracy demands higher resolution. This split-resolution strategy keeps accuracy intact where it matters while saving significant compute on the landmark and pose models.

The results comparison:

Table 5. Baseline and Final System Performance Comparison

Metric	Target	Baseline (pre-opt.)	Final Result
Latency (p95)	< 500 ms	100.2 ms (sequential)	< 62.5ms
FPS	> 10 FPS	4–7 FPS	~30 FPS
Power consumption	< 25 W	< 8 W	< 8 W
CPU usage	< 80%	< 77%	< 77%
GPU usage	< 100%	< 98%	< 98%

VI. System Performance Results

Final numbers were measured on the Jetson Orin Nano Super under sustained load using tegrastats and our internal logging hooks like calculating Precision, Recall and F1 score. Accuracy Metrics (Hold-out Validation Video):

1. **Recall:** 100%
2. **Precision:** 93.3%
3. **F1-Score:** 96.5%

Latency & Resource Table (TensorRT FP16):

Table 6. Final Accuracy, Latency, and Resource Utilization Metrics

Metric	Baseline	Final Shipped System
FPS	4 – 7	20.0
p50 Latency (Median)	—	48.9 ms
p95 Latency	—	52.4 ms
p99 Latency (Tail)	100.2 ms	62.5 ms
CPU Usage	77%	71%
GPU Usage	98%	64%

Metric	Baseline	Final Shipped System
Power (VDD_IN)	7.6 W	6.8 W

VII. Lessons Learned

Implementing this system in a real embedded environment exposed several unexpected challenges that textbook knowledge alone could not have prepared us for. Below are the five most impactful lessons from this project.

- A. The CI/CD Paradigm Shift: Because this was our first time implementing a Continuous Integration and Continuous Deployment pipeline. Setting up GitHub Actions to enforce strict quality gates such as requiring $\geq 90\%$ Pytest coverage, blocking merges on security vulnerabilities (via Bandit/Pip Audit), and automating ARM64 Docker builds fundamentally changed how we treated code quality. Furthermore, configuring a self-hosted Jetson runner for automated deployment taught us the immense value of validating release candidates directly on the target hardware to prevent architecture mismatches.
- B. Low-Level Hardware Pin Constraints: We encountered unexpected embedded-systems roadblocks when interfacing with physical hardware, specifically failing to access the pins for our yellow and red LEDs. Standard libraries couldn't control them because Jetson's OS did not expose those specific pins as standard GPIO by default. To resolve this, we had to bypass the standard abstractions and utilize a busybox workaround to manually reconfigure those specific pins as functional GPIO. This was a stark reminder that deploying edge devices requires troubleshooting at the system and hardware register level, not just the application level.
- C. Fixed Thresholds Fail in the Real World: We initially proposed a fixed EAR threshold. During testing, we learned quickly that variations in human eye geometry completely break static thresholds. Building a personalized calibration sequence was much harder than expected but necessary for a production system.
- D. Asynchronous Audio in Python is Hostile: We discovered that standard audio libraries (like sounddevice) easily block the main thread or fail silently on edge hardware. We had to engineer custom logic to generate synthetic .wav files and actively manage the PID of the Linux aplay utility to achieve instant, non-blocking hardware alarms.
- E. PyCUDA Contiguous Memory Constraints: TensorRT fails silently or outputs garbage data if the NumPy arrays fed into the GPU are not contiguous in memory. We learned the hard way that transposing axes (`transpose(2, 0, 1)`) breaks memory contiguity, requiring a strict `np.ascontiguousarray()` cast before inference.

VIII. What We'd Do Differently if We Did It Again

Although the project successfully met its technical objectives, the implementation process revealed several decisions that we would approach differently if starting from scratch. Reflecting on these

experiences helped identify opportunities to reduce development time, improve evaluation efficiency, and better utilize available hardware resources.

- A. Research and Implement EAR Calibration on Day One: Because we initially relied on a fixed threshold, we had to heavily refactor the state machine late in the project to accommodate dynamic personalized baselines. Choosing this dynamic target on day one would have saved a week of rework.
- B. Automate the Evaluation Framework Immediately: Ground-truth labeling of video data second-by-second involved immense manual effort. Next time, we would build scripts to automate ground-truth verification from the start to speed up model tuning and testing reproducibility.
- C. Expand Scope from the Start: With the GPU headroom we successfully recovered via TensorRT and Async threading (dropping GPU usage from 98% to 64%), we had the computed budget to monitor secondary behaviors like seatbelt usage or smoking detection, which we would integrate early in a second iteration.

IX. Individual Reflections

Farhan Hikmatullah Daulay: Led the development of the core AI pipeline, Jetson hardware deployment, and system architecture. Managed the integration of YOLOv26n, 6DRepNet360, and PFLD. Built out the asynchronous threading model to ensure the code could reliably run on the physical edge device without blocking the camera feed. Learned a massive amount about edge optimization, specifically how moving from synchronous PyTorch to threaded FP16 engines fundamentally changes resource consumption.

Kishore Sridhar: Led the development of the rigorous test-driven infrastructure and hardware integration. Engineered the comprehensive pytest suite, achieving over 90% code coverage across all modules by implementing advanced mocking strategies to simulate Jetson GPIO, compiling the models into TensorRT engines and handling the complex PyCUDA memory alignment and asynchronous camera I/O without requiring physical hardware. Worked with Docker and designed the continuous integration of pipeline quality gates and managed the system's power-profile configurations (nvpmodel). Learned the critical importance of isolating background threads during unit testing to prevent CI pipeline hangs and gained deep experience in hardware-software interfacing using automated deployment scripts in CD pipelines.

X. Acknowledgments & References:

A. Acknowledgments

1. The team would like to express sincere gratitude to Prof. Cho Li, Prof. Hsieh, and Prof. J.L. Sung for their guidance, valuable feedback, and continuous support throughout the capstone project.
2. We also thank Mr. Dung Dinh Trong (TA) for his technical assistance and helpful suggestions during system development and testing.

3. Finally, we thank Billy for generously providing the USB speaker used during hardware integration, testing, and project demonstrations.

B. References

- [1] X. Guo, S. Li, J. Yu, J. Zhang, J. Ma, L. Ma, W. Liu, and H. Ling, "PFLD: A Practical Facial Landmark Detector," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, 2019, pp. 3824–3832.
- [2] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs," arXiv:1907.05047, Jul. 2019.
- [3] Y. Hempel, A. Abdelrahman, and A. Al-Hamadi, "6DRepNet: End-to-End 6DoF Head Pose Estimation from Monocular RGB Images," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, Bordeaux, France, 2022, pp. 783–787.
- [4] G. Jocher et al., "Ultralytics YOLO Documentation," *Ultralytics*, 2025. [Online]. Available: <https://docs.ultralytics.com>
- [5] NVIDIA Corporation, "NVIDIA TensorRT Developer Guide," *NVIDIA*, 2025. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt>
- [6] NVIDIA Corporation, "Jetson Orin Nano Developer Kit User Guide," *NVIDIA*, 2025. [Online]. Available: <https://developer.nvidia.com/embedded>
- [7] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–126, 2000.
- [8] Docker Inc., "Docker Documentation," *Docker*, 2025. [Online]. Available: <https://docs.docker.com>
- [9] GitHub Inc., "GitHub Actions Documentation," *GitHub*, 2025. [Online]. Available: <https://docs.github.com/actions>
- [10] pytest Development Team, "pytest Documentation," 2025. [Online]. Available: <https://docs.pytest.org>
- [11] Python Software Foundation, "Python Queue Module Documentation," 2025. [Online]. Available: <https://docs.python.org/3/library/queue.html>
- [12] NVIDIA Corporation, "Jetson.GPIO Documentation," *NVIDIA*, 2025. [Online]. Available: <https://github.com/NVIDIA/jetson-gpio>
- [13] A. Klöckner et al., "PyCUDA and PyOpenCL: A Scripting-Based Approach to GPU Run-Time Code Generation," *Parallel Computing*, vol. 38, no. 3, pp. 157–174, 2012.